

GraphBench Challenge

Réfutation automatique de conjectures en théorie des graphes

GOUASMI Yassine
MOKHTARY Moumen Aya

Université Paris Nanterre
Master 1 MIAGE
Optimisation Combinatoire
Enseignant : François Delbot

09/05/2026

Dépôt GitHub :

<https://github.com/YassineGOUASMI/GraphBench-Project>

Résultat final : 20/20 contre-exemples trouvés

Score final approximatif : 35.52

Table des matières

1	Introduction	3
2	Architecture du projet	3
2.1	Structure générale	3
2.2	Rôle des principaux fichiers	3
3	Génération et mutations des graphes	4
3.1	Génération initiale	4
3.2	Mutations locales	4
4	Calcul des invariants	4
4.1	Invariants utilisés	4
5	Recherche évolutionnaire	5
5.1	Principe général	5
5.2	Critère de réussite	5
6	Recherche baseline	5
6.1	Extrait du moteur baseline	5
6.2	Capture d'exécution	6
7	Amélioration inspirée de FunSearch	6
7.1	Score heuristique	7
7.2	Sélection mixte	7

8 Résultats expérimentaux	7
8.1 Résultat final	7
8.2 Résumé	8
8.3 Capture du résultat final	8
9 Limites	8
10 Discussion scientifique	8
11 Conclusion	9

1 Introduction

Le challenge GraphBench consiste à rechercher automatiquement des contre-exemples à des conjectures en théorie des graphes.

L'idée générale du projet est de générer des graphes, de calculer plusieurs invariants, puis de vérifier si certaines conjectures du benchmark sont violées.

Pour cela, nous avons développé un programme capable de :

- générer des graphes aléatoires,
- modifier ces graphes avec différentes mutations,
- calculer leurs invariants,
- évaluer la violation d'une conjecture,
- conserver les meilleurs graphes trouvés.

Le système obtenu permet finalement de trouver des contre-exemples pour les 20 premières conjectures testées du benchmark.

2 Architecture du projet

Le projet est organisé en plusieurs modules spécialisés.

2.1 Structure générale

```
graphbench-project/  
|-- benchmark/  
| |-- benchmark.xlsx  
|-- experiments/  
| |-- run_basic_search.py  
| |-- run_baseline_search.py  
| |-- run_final.py  
|-- results/  
| |-- basic_search/  
| |-- baseline_search/  
| |-- final/  
|-- src/  
| |-- conjecture.py  
| |-- generator.py  
| |-- invariants.py  
| |-- mutations.py  
| |-- search.py  
| |-- search_baseline.py  
| |-- scoring.py  
|-- requirements.txt  
|-- README.md
```

2.2 Rôle des principaux fichiers

- `generator.py` : génération initiale des graphes,
- `mutations.py` : opérateurs de mutation,
- `invariants.py` : calcul des invariants,
- `search.py` : moteur de recherche guidé par score heuristique,
- `search_baseline.py` : baseline classique,
- `scoring.py` : fonction de score inspirée de FunSearch,
- `run_basic_search.py` : expérience principale avec score heuristique,
- `run_baseline_search.py` : expérience baseline,

— `run_final.py` : génération des résultats finaux.

3 Génération et mutations des graphes

3.1 Génération initiale

Le système génère des graphes aléatoires connexes servant de population initiale.
Les tailles de graphes sont variées afin de découvrir différentes régions de l'espace de recherche.

3.2 Mutations locales

Les opérateurs de mutation implémentés sont :

- ajout d'arête,
- suppression d'arête,
- ajout de sommet,
- suppression de sommet,
- ajout de feuille,
- subdivision d'arête,
- ajout de chemin,
- ajout de clique,
- ajout de faux jumeau,
- densification locale.

Ces mutations permettent d'explorer progressivement des graphes de structures variées.

4 Calcul des invariants

Le moteur calcule de nombreux invariants de graphes à l'aide de NetworkX.

4.1 Invariants utilisés

- nombre de sommets,
- nombre d'arêtes,
- degré minimum,
- degré maximum,
- degré moyen,
- diamètre,
- rayon,
- nombre de triangles,
- clique maximum,
- nombre d'indépendance,
- couverture par sommets,
- couplage maximum,
- domination,
- domination indépendante,
- domination totale,
- connectivité par sommets,
- connectivité par arêtes.

5 Recherche évolutionnaire

5.1 Principe général

Le système maintient une population de graphes candidats.

À chaque itération :

1. un graphe parent est sélectionné,
2. une mutation est appliquée,
3. les invariants sont recalculés,
4. la violation de la conjecture est évaluée.

Les meilleurs graphes sont conservés dans la population.

Aucun contre-exemple connu n'a été codé manuellement dans le système. Les graphes sont générés et modifiés automatiquement par mutations successives.

5.2 Critère de réussite

Une conjecture est réfutée lorsque :

$$\text{violation}(G) > 0$$

Le graphe correspondant devient alors un contre-exemple valide.

6 Recherche baseline

La baseline utilise directement la violation réelle comme score principal.

6.1 Extrait du moteur baseline

```
score = conjecture.violation(invariants)

if score > best_score:
    best_score = score
```

Cette approche a servi de point de comparaison. Elle donne déjà de bons résultats, mais certaines conjectures restent plus difficiles selon les exécutions.

6.2 Capture d'exécution

```
--- Restart 1/3 ---
Score initial : -0.75
Nouveau meilleur score : -0.5
Nouveau meilleur score : -0.25
Nouveau meilleur score : 0.0
Nouveau meilleur score : 0.25

===== MEILLEUR RÉSULTAT BASELINE =====
Trouvé : True
Score : 0.25
Temps : 2.04 sec
Coût : 2.04

=====
Conjecture 2117
If G is a connected graph,  $x = \text{matching\_number}$  and  $y = \text{independent\_}$ 
 $7 * x(G)^2 + 1/3 * x(G)^3$ 
=====

--- Restart 1/3 ---
Score initial : -3.2275714285714194
Nouveau meilleur score : -2.2275714285714194
Nouveau meilleur score : -1.2275714285714194
Nouveau meilleur score : -0.5132857142857077
Nouveau meilleur score : 0.4867142857142923

===== MEILLEUR RÉSULTAT BASELINE =====
Trouvé : True
Score : 0.4867142857142923
Temps : 0.54 sec
Coût : 0.54
```

FIGURE 1 – Exécution de la recherche baseline

7 Amélioration inspirée de FunSearch

Après la première baseline, nous avons amélioré la recherche avec une fonction de score plus informative. L'idée est proche de FunSearch : au lieu de classer les graphes uniquement avec la violation réelle, on ajoute un score heuristique qui aide à choisir des graphes prometteurs.

La fonction finale utilisée ne cible aucune conjecture par son identifiant. Elle est générale et dépend seulement :

- de la violation réelle,
- des invariants du graphe,
- du signe de la conjecture,
- des variables x et y présentes dans la formule.

7.1 Score heuristique

La fonction de score donne un poids très important à la violation réelle, puis ajoute de petits bonus ou pénalités selon les invariants du graphe. Cela permet de ne pas s'éloigner de l'objectif principal tout en guidant la recherche.

```
score = 1000.0 * violation

if conjecture.sign == "<=":
    score += 0.2 * y
    score -= 0.05 * x
elif conjecture.sign == ">=":
    score -= 0.2 * y
    score += 0.05 * x

score += 0.03 * triangles
score += 0.15 * clique
score += 0.05 * domination
score += 0.05 * total_domination
score -= 0.002 * n
```

7.2 Sélection mixte

La recherche conserve ensuite deux types de bons candidats :

- les graphes ayant la meilleure violation réelle,
- les graphes ayant le meilleur score heuristique.

```
top_real = sorted(population, key=lambda x: x[0], reverse=True)[:20]
top_heuristic = sorted(population, key=lambda x: x[1], reverse=True)[:20]

population = top_real + top_heuristic
```

Cette sélection mixte permet de garder une partie exploratoire dans la population, tout en continuant à privilégier les vrais contre-exemples. Cette version a permis d'obtenir les meilleurs résultats sans utiliser de recherche ciblée par identifiant de conjecture.

8 Résultats expérimentaux

8.1 Résultat final

Les expériences ont été réalisées sur les 20 premières conjectures du benchmark, afin de conserver un temps d'exécution raisonnable.

Le système a finalement obtenu :

20 / 20 contre-exemples trouvés

8.2 Résumé

Mesure	Valeur
Conjectures testées	20
Contre-exemples trouvés	20
Taux de réussite	100%
Coût approximatif	35.52
Temps moyen	1.78 sec

TABLE 1 – Résumé final des performances

8.3 Capture du résultat final

```
Résumé sauvegardé dans results/basic_search/summary.txt
(venv) PS C:\Users\Ce Pc\Documents\graphbench-project> python experiments/run_final.py

===== FINAL RESULTS =====
Conjectures tested : 20
Counterexamples found : 20
Success rate : 100.0 %
Approximate total cost : 35.52
Average time : 1.78 sec

Saved:
results/final/final_results.csv
results/final/final_summary.txt
(venv) PS C:\Users\Ce Pc\Documents\graphbench-project> |
```

FIGURE 2 – Résultat final obtenu avec la recherche inspirée de FunSearch

9 Limites

- dépendance à l'aléatoire,
- coût croissant pour les conjectures difficiles,
- absence d'apprentissage automatique réel,
- la fonction de score reste écrite manuellement et n'est pas encore générée automatiquement.

10 Discussion scientifique

Les conjectures les plus faciles à réfuter ont principalement concerné les invariants locaux ou combinatoires tels que le nombre de triangles, le degré maximum, la clique maximum ou le couplage. Ces invariants réagissent fortement aux mutations locales comme l'ajout de clique ou la densification locale.

Les conjectures les plus difficiles ont concerné des invariants plus globaux comme la domination totale, la domination indépendante et la couverture par sommets. Ces valeurs évoluent moins directement lorsqu'on ajoute ou supprime une seule arête.

Les mutations les plus efficaces ont été l'ajout de clique, la densification locale, l'ajout de faux jumeaux et l'ajout de chemins. Elles permettent de modifier rapidement plusieurs invariants tout en conservant souvent la connexité du graphe.

L'approche inspirée de FunSearch a amélioré la recherche finale grâce à une sélection mixte entre violation réelle et score heuristique. Elle a permis d'obtenir les 20 contre-exemples sans utiliser de ciblage manuel des conjectures.

L'utilisation de l'IA générative a surtout été utile pour proposer des structures de code, organiser les expériences, générer des variantes de score et accélérer le développement. Cependant, les meilleures performances ont été obtenues après plusieurs essais et ajustements manuels.

11 Conclusion

Ce projet a permis de construire un moteur complet de recherche automatique de contre-exemples en théorie des graphes.

Les principaux apports sont :

- une architecture modulaire,
- un moteur évolutionnaire robuste,
- plusieurs stratégies de recherche,
- une amélioration de la recherche par score heuristique inspiré de FunSearch.

Les résultats obtenus montrent qu'une approche évolutionnaire guidée par une fonction de score peut produire de très bons résultats sur un benchmark de conjectures.

De futures améliorations pourraient inclure :

- la génération automatique de nouvelles fonctions de score,
- des heuristiques adaptatives,
- des mutations guidées par les invariants,
- des modèles génératifs de graphes.